Word Embeddings for Sentiment Classification

Comparing Word2Vec, GloVe, and fastText CBOW with an LSTM

Assignment purpose. Build one controlled sentiment-classification pipeline in Python, replace only the word-embedding component, and use the results to explain how word-level context, global co-occurrence, and subword information affect an LSTM.

Assignment snapshot

Item	Requirement
Public dataset	IMDB Large Movie Review Dataset: 25,000 labeled training reviews and 25,000 labeled test reviews.
Prediction task	Binary sentiment classification: positive versus negative movie review.
Embedding comparison	Word2Vec CBOW, GloVe, and fastText CBOW, plus a random-embedding control.
Downstream model	The same unidirectional LSTM architecture for every embedding condition.
Primary output	A reproducible comparison of classification quality, vocabulary coverage, training cost, and failure cases.

1 Learning objectives

- Explain the difference between word-level and subword-level representations.
- Implement an embedding matrix that maps token IDs to dense vectors and can be loaded into a PyTorch Embedding layer.
- Train or load Word2Vec, GloVe, and fastText CBOW representations while avoiding label leakage.
- Use the same LSTM classifier, data split, optimization settings, and evaluation protocol for a fair comparison.
- Interpret performance differences using coverage, semantic similarity, OOV behavior, training curves, and error analysis.

2 Problem statement

A movie review is a sequence of words whose sentiment must be classified as positive or negative. Your task is to test whether the way words are represented before they enter an LSTM changes the final classification result. The central experimental rule is simple: keep the sentiment dataset, tokenization policy, LSTM architecture, optimizer, and evaluation procedure as constant as possible; change the embedding source.

Important terminology. Use GloVe as the name of the global co-occurrence method. In this assignment, Word2Vec and fastText are both configured with CBOW (continuous bag of words). GloVe is not a CBOW model; it learns vectors from a global word co-occurrence matrix.

3 Embeddings to compare

Embedding	Main learning signal	Representation unit	Expected strength	Main caveat
Word2Vec CBOW	Predict a target word from nearby context words.	Whole words	Efficient local semantic relationships.	An unseen word has no learned word vector.

Embedding	Main learning signal	Representation unit	Expected strength	Main caveat
GloVe	Fit vectors to global word co-occurrence statistics.	Whole words	Captures broad corpus-level relationships.	Coverage depends on the external vocabulary and corpus.
fastText CBOW	CBOW context prediction plus character n-grams.	Words and subwords	Handles rare, morphologically related, and misspelled forms more gracefully.	Character n-grams can introduce noisy similarities and extra computation.

4 Data and experimental controls

Dataset. Use the public IMDB dataset through Hugging Face Datasets with load_dataset("imdb"). The official training split contains 25,000 labeled reviews; the official test split contains another 25,000. Create an 80/20 train-validation split from the training split and keep the official test split untouched until the final evaluation.

Preprocessing. Write one deterministic tokenizer and use it for all three embedding conditions. Lowercase text, keep alphabetic tokens and apostrophes, build the vocabulary from training reviews only, reserve <pad> and <unk>, and pad or truncate each review to a maximum length of 300 tokens.

Leakage rule. Do not use sentiment labels, official test reviews, or test-derived vocabulary statistics when training Word2Vec or fastText. If you use an external GloVe file, record its source and explain that its training corpus differs from the IMDB corpus.

Fairness rule. Use the same vocabulary, embedding dimension, sequence length, train-validation split, batch size, LSTM architecture, optimizer, learning-rate schedule, early-stopping rule, and random seeds across the primary runs.

5 Required software and project structure

Use Python 3.10 or later. The recommended stack is PyTorch for the LSTM, Gensim for Word2Vec and fastText, Hugging Face Datasets for loading IMDB, scikit-learn for metrics, and Matplotlib for plots.

```
project/
  README.md
  requirements.txt
  src/
    data.py             # loading, tokenization, vocabulary, padding
    embeddings.py       # Word2Vec, GloVe, fastText, matrix alignment
    model.py            # LSTM classifier
    train.py            # training loop and checkpointing
    evaluate.py         # metrics, plots, error analysis
  results/
    metrics.csv
    learning_curves.png
    confusion_matrices.png
```

6 Required implementation pipeline

1. Load the IMDB dataset and create a reproducible train-validation split. Save the split indices or document the random seed.
2. Tokenize the reviews and build a vocabulary using training text only. Report the vocabulary size and the percentage of tokens mapped to <unk> in validation and test data.
3. Train a Word2Vec CBOW model on the tokenized training reviews. Use a fixed dimension of 100, window size of 5, min_count of 2, negative sampling, and at least 10 training epochs unless you justify different settings.
4. Load GloVe vectors with dimension 100. You may use GloVe 100-dimensional vectors from a public source such as glove-wiki-gigaword-100 or a downloaded glove.6B.100d file. Record the exact source in README.md.
5. Train a fastText CBOW model on the same tokenized training reviews. Use dimension 100, window size of 5, min_count of 2, and character n-grams with min_n = 3 and max_n = 6 unless you justify different settings.

6. Align every embedding source to the same vocabulary and create an embedding matrix with shape [vocab_size, 100]. Initialize missing Word2Vec and GloVe rows with a small random distribution; set the <pad> row to zeros. For fastText, report both vocabulary coverage and the ability to generate a vector for an unseen token from subwords.
7. Implement one unidirectional LSTM classifier and reuse it for every embedding condition. The primary model should use hidden size 128, one LSTM layer, a linear output head with one logit, BCEWithLogitsLoss, Adam with learning rate 1e-3, batch size 64, and early stopping with patience 2.
8. Run the random-embedding control, Word2Vec CBOW, GloVe, and fastText CBOW under the same training protocol. Then repeat the three embedding experiments with the embedding layer trainable as an extension.

7 Model interface and starter skeleton

The forward path should be conceptually: token IDs -> embedding lookup -> LSTM -> last valid hidden state -> linear layer -> sentiment logit. Use sequence lengths or packed sequences so that padding does not become a learned signal. The following interface is a starting point, not a complete solution.

```
class SentimentLSTM(nn.Module):
    def __init__(self, embedding_matrix, hidden_dim=128, trainable=False):
        super().__init__()
        # TODO: create nn.Embedding from the matrix
        # TODO: set requires_grad according to trainable
        # TODO: create one unidirectional nn.LSTM
        # TODO: create a linear layer that returns one logit
        pass

    def forward(self, token_ids, lengths):
        # TODO: mask or pack padded sequences
        # TODO: return one logit per review
        pass

def build_embedding_matrix(vocab, vector_lookup, dim=100):
    # TODO: start with a small random matrix
    # TODO: copy vectors for tokens found in vector_lookup
    # TODO: set the <pad> row to zeros and return coverage stats
    pass
```

8 Required experiments

Run	Embedding condition	Embedding layer	Purpose
E0	Random 100-dimensional vectors	Frozen or fixed control	Measure what the LSTM learns without external semantic initialization.
E1	Word2Vec CBOW trained on IMDB training text	Frozen	Test word-level local context representations.
E2	GloVe 100-dimensional public vectors	Frozen	Test global co-occurrence representations from an external corpus.
E3	fastText CBOW trained on IMDB training text	Frozen	Test the value of character n-grams and subword information.
E4	Word2Vec, GloVe, and fastText repeated	Trainable	Measure how much task-specific fine-tuning changes the comparison.

Minimum evidence. For E0-E3, report results from at least three random seeds if compute allows. If you use one seed for a pilot, state that limitation and use three seeds for the final comparison. Do not tune one embedding method more aggressively than the others.

9 Evaluation and analysis requirements

- Report accuracy, precision, recall, F1-score, macro-F1, and ROC-AUC on the untouched official test set.
- Show training and validation loss curves, plus a confusion matrix for each primary embedding condition.

- Report mean and standard deviation across seeds when multiple seeds are used.
- Measure token-level coverage and unique-vocabulary coverage for each embedding source. Include the percentage of validation and test tokens that become <unk>.
- Report training time, number of trainable parameters, and peak memory if available. Explain whether the runtime differences are meaningful for this task.
- Perform an error analysis on at least 10 misclassified reviews. Group errors into themes such as negation, sarcasm, long-range dependencies, rare words, spelling variation, or ambiguous sentiment.

10 Subword and OOV investigation

fastText is expected to offer an advantage when a token is rare, morphologically related to known words, or absent from its explicit word vocabulary. Demonstrate this rather than assuming it. First, compare coverage statistics. Then choose one of the following investigations:

- Required option: create a small evaluation subset containing rare or unseen tokens and compare the performance of the frozen fastText model with the other frozen models.
- Extension option: implement a dynamic fastText lookup for tokens outside the training vocabulary using fasttext_model.wv.get_vector(token), and compare it with the <unk> fallback.
- Extension option: create a controlled spelling-variation set by applying a documented character edit to selected review words, then report how performance changes.

Interpretation caution. If fastText is trained on the same small IMDB corpus while GloVe is loaded from a much larger external corpus, do not attribute every difference to subword information. Discuss corpus size, domain, training objective, and coverage as separate factors.

11 Questions your report must answer

1. Which embedding produces the strongest test macro-F1, and is the difference large enough to matter across random seeds?
2. Does the ranking of Word2Vec, GloVe, and fastText change when the embedding layer is trainable? Why might fine-tuning reduce or increase the value of the initial vectors?
3. How do vocabulary coverage and <unk> rates relate to classification performance? Is higher coverage always better?
4. What evidence supports or challenges the claim that fastText helps with rare words, morphology, or spelling variation?
5. What kinds of reviews are misclassified by all models? What does this reveal about the limits of word embeddings and a one-layer LSTM?
6. If you had to deploy one embedding for a new sentiment dataset in a different domain, which would you choose and what additional validation would you require?

12 Deliverables

- A runnable Python project or notebook that reproduces the main experiments from a clean environment.
- A README.md with installation steps, data and embedding sources, commands, hardware information, and expected output files.
- A results.csv containing metrics for every run, seed, embedding condition, and trainable/frozen setting.
- Figures for learning curves and confusion matrices, with readable labels and captions.
- A short technical report of 4 to 6 pages answering the required questions and discussing limitations.
- A statement of any external files, pretrained vectors, code references, or generative tools used.

Optional extensions

- Compare CBOW with Word2Vec Skip-gram while keeping dimension and vocabulary fixed.
- Compare unidirectional and bidirectional LSTMs, then discuss whether the comparison remains controlled.
- Train a small GloVe model on the same IMDB training text and compare same-corpus versus external-corpus initialization.
- Add a simple attention layer and analyze whether it changes error patterns, not only the final score.
- Test transfer to another public sentiment dataset and discuss domain shift.